

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**
Факультет информационных технологий
Кафедра параллельных вычислений

ОТЧЕТ

О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ

**«РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ
ИТЕРАЦИОННЫМИ МЕТОДАМИ»**

студента 2 курса, 24203 группы

Анисимова Льва Евгеньевича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
доцент кафедры параллельных
вычислений
Андрей Юрьевич Власенко

Новосибирск, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
Приложение 1. Код программы gen.cpp.....	6
Приложение 2. Код программы sequential.cpp.....	7
Приложение 3. Код программы mpi.cpp.....	9
Приложение 4. Код скрипта запуска на кластере.....	11
Приложение 5. Таблицы.....	12
Приложение 6. Графики.....	13
Приложение 7. Профилирование.....	15

ЦЕЛЬ

Изучение принципов разработки параллельных алгоритмов с использованием интерфейса передачи сообщений MPI. Освоение методов декомпозиции данных (разрезание матриц и векторов) и коллективного взаимодействия процессов для эффективного решения систем линейных алгебраических уравнений (СЛАУ) итерационными методами.

ЗАДАНИЕ

Вариант: №2 — Метод минимальных невязок.

Формулировка задачи:

I. Реализовать параллельную версию метода минимальных невязок для решения СЛАУ вида $Ax = b$.

Алгоритм: На каждой итерации вычисляются невязка $y^n = Ax^n - b$, итерационный параметр $\tau^n = \frac{(y^n, Ay^n)}{(Ay^n, Ay^n)}$ и новое приближение $x^{n+1} = x^n - \tau^n y^n$.

Параллельная схема: Матрица A разрезается по строкам между MPI-процессами. Реализуется второй вариант распределения данных: векторы x , y и b также разрезаются между процессами аналогично матрице.

Критерий остановки: $\frac{\|Ax^n - b\|_2}{\|b\|_2} < \epsilon$, где $\epsilon = 10^{-5}$.

II. Составить отчет, содержащий исходные коды разработанных программ и построенные графики.

ОПИСАНИЕ РАБОТЫ

В рамках выполнения лабораторной работы на языке C++ разработан и откомпилирован исходный код (Приложение 1), реализующий параллельный алгоритм решения системы линейных алгебраических уравнений (СЛАУ) методом минимальных невязок.

Экспериментальным путем подобран оптимальный размер размерности матрицы $N = N_y \times N_x$ и параметр точности $\epsilon = 10^{-5}$, при которых время работы последовательной программы на одном вычислительном узле составляет не менее 30 секунд.

Реализована программная среда, использующая библиотеку MPI для организации параллельных вычислений по схеме с разрезанием матрицы A и векторов x, y, b на блоки строк, распределяемые между процессами (Приложение 2), что соответствует второму варианту задания.

Взаимодействие между процессами внутри итерационного цикла организовано с использованием коллективных операций: передача частей вектора для обеспечения полноты данных при умножении матрицы на вектор осуществляется функцией `MPI_Allgatherv`, а вычисление глобальных скалярных произведений для определения итерационного параметра τ^n и проверки критерия сходимости — функцией `MPI_Allreduce`.

Проведены замеры времени выполнения реализации при использовании 1, 2, 4, 8, 16 и 24 процессорных ядер, при этом для каждой конфигурации выполнено несколько запусков с целью выбора минимального времени и исключения влияния случайных факторов.

На основании полученных данных выполнен расчет показателей ускорения и эффективности распараллеливания, результаты которых сведены в таблицу (Приложение 3), а визуализация зависимостей представлена в виде графиков (Приложение 4).

Дополнительно проведено профилирование программы с помощью библиотеки MPE и инструмента Jumpshot для анализа временных затрат на коммуникации между процессами (Приложение 5).

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была реализована параллельная версия метода минимальных невязок для решения СЛАУ с использованием интерфейса MPI. Экспериментальные данные подтвердили, что увеличение числа вычислительных узлов обеспечивает существенное ускорение работы программы до определенного предела. Однако анализ профилирования в инструменте Jumpshot показал, что итерационный процесс характеризуется высокой плотностью коллективных взаимодействий, занимающих значительную долю общего времени выполнения.

Основной вклад в накладные расходы вносят операции `MPI_Allgather` (более 1800 вызовов) и `MPI_Allreduce` (более 5400 вызовов), что обусловлено математической структурой метода минимальных невязок, требующего двукратного вычисления скалярных произведений и полной сборки вектора x для выполнения матрично-векторного умножения на каждой итерации. На временной диаграмме отчетливо визуализируется чередование вычислительных фаз и фаз синхронизации, причем с ростом числа процессов до 24 время, затрачиваемое на ожидание и пересылку данных, начинает сопоставляться с временем полезных вычислений. Дополнительным фактором, снижающим эффективность на начальном этапе, является дисковый ввод-вывод (`MPI_File_read_at`), занимающий существенный интервал времени при инициализации данных.

На основании построенных графиков времени, ускорения и эффективности можно сделать вывод о целесообразности использования данной параллельной реализации на малых и средних конфигурациях (до 16 процессов). Дальнейшее масштабирование ограничивается возрастающей сложностью коммуникационной среды, что подтверждает классическую зависимость снижения параллельной эффективности при увеличении числа занятых ядер для задач с высокой связностью данных.

Приложение 1. Код программы gen.cpp

```
#include <vector>
#include <random>
#include <fstream>
#include <iostream>
#include <string>

using namespace std;

int main() {
    int n = 1000;
    string mat_file = "matrix.bin";
    string vec_file = "vector_b.bin";

    mt19937 gen(42);
    uniform_real_distribution dist(-1.0, 1.0);
    uniform_real_distribution dist_rhs(-1.0, 1.0);

    vector<double> R(n * n);
    vector<double> A(n * n, 0.0);
    vector<double> b(n);

    cout << "Генерация системы " << n << "x" << n << " (улучшенная
    версия)..." << endl;

    // 1. Генерируем вспомогательную матрицу R
    for (auto &v : R) v = dist(gen);

    // 2.  $A = R^T * R + 0.5 * I$ 
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            for (int k = 0; k < n; ++k) {
                A[i * n + j] += R[k * n + i] * R[k * n + j];
            }
        }
        A[i * n + i] += 0.5;           // eps = 0.5
    }

    // 3. b
    for (auto &v : b) v = dist_rhs(gen);

    ofstream f_mat(mat_file, ios::binary);
    f_mat.write(reinterpret_cast<char*>(A.data()), A.size() *
    sizeof(double));
    f_mat.close();

    ofstream f_vec(vec_file, ios::binary);
    f_vec.write(reinterpret_cast<char*>(b.data()), b.size() *
    sizeof(double));
    f_vec.close();

    cout << "Файлы готовы: " << mat_file << ", " << vec_file << endl;
    return 0;
}
```

Приложение 2. Код программы *sequential.cpp*

```
#include <iostream>
#include <vector>
#include <fstream>
#include <cmath>
#include <iomanip>
#include <chrono>

constexpr int N = 6960;
constexpr double EPS = 1e-12;
constexpr int MAX_ITER = 10000;

int main()
{
    auto t_start = std::chrono::high_resolution_clock::now();

    std::vector<double> A(N * N);
    std::vector<double> b(N);

    // Загрузка матрицы
    std::ifstream fa("matrix_a.bin", std::ios::binary);
    if (!fa)
    {
        std::cerr << "Error opening matrix_a.bin\n";
        return 1;
    }
    fa.read(reinterpret_cast<char*>(A.data()), N * N * sizeof(double));
    fa.close();

    // Загрузка вектора
    std::ifstream fb("vector_b.bin", std::ios::binary);
    if (!fb)
    {
        std::cerr << "Error opening vector_b.bin\n";
        return 1;
    }
    fb.read(reinterpret_cast<char*>(b.data()), N * sizeof(double));
    fb.close();

    std::vector<double> x(N, 0.0);
    std::vector<double> r(N), Ar(N);

    double r0_norm = 0.0;
    for (double val : b) r0_norm += val * val;
    r0_norm = std::sqrt(r0_norm);

    int converged_iter = -1;
    for (int iter = 0; iter < MAX_ITER; ++iter)
    {
        // r = b - Ax
        for (int i = 0; i < N; ++i)
        {
            double sum = 0.0;
            for (int j = 0; j < N; ++j) sum += A[i * N + j] * x[j];
            r[i] = b[i] - sum;
        }

        double r_norm2 = 0.0;
        for (double val : r) r_norm2 += val * val;
        double r_norm = std::sqrt(r_norm2);

        if (r_norm / r0_norm < EPS)
        {
```

```

        converged_iter = iter;
        break;
    }

    // tau = (r, Ar) / (Ar, Ar)
    for (int i = 0; i < N; ++i)
    {
        Ar[i] = 0.0;
        for (int j = 0; j < N; ++j) Ar[i] += A[i * N + j] * r[j];
    }

    double rAr = 0.0, ArAr = 0.0;
    for (int i = 0; i < N; ++i)
    {
        rAr += r[i] * Ar[i];
        ArAr += Ar[i] * Ar[i];
    }

    double tau = rAr / ArAr;
    for (int i = 0; i < N; ++i) x[i] += tau * r[i];
}

auto t_end = std::chrono::high_resolution_clock::now();
std::cout << "Time: " << std::chrono::duration<double>(t_end -
t_start).count() << "s\n";
std::cout << "Converged: " << converged_iter << ", x[0]: " << x[0] <<
"\n";

return 0;
}

```


Приложение 3. Код программы *tri.cpp*

```
#include <mpi.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>

constexpr int N = 6960;
constexpr double EPS = 1e-12;
constexpr int MAX_ITER = 10000;

int main(int argc, char** argv)
{
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    double t_start = MPI_Wtime();

    int local_N = N / size;
    int local_start = rank * local_N;

    // Чтение локальной части матрицы
    std::vector<double> A_local(local_N * N);
    MPI_File fh_a;
    MPI_File_open(MPI_COMM_WORLD, "matrix_a.bin", MPI_MODE_RDONLY,
MPI_INFO_NULL, &fh_a);
    MPI_File_read_at(fh_a, local_start * N * sizeof(double), A_local.data(),
local_N * N, MPI_DOUBLE,
MPI_STATUS_IGNORE);
    MPI_File_close(&fh_a);

    // Чтение всего вектора B (он нужен всем для расчета r_local)
    std::vector<double> b(N);
    MPI_File fh_b;
    MPI_File_open(MPI_COMM_WORLD, "vector_b.bin", MPI_MODE_RDONLY,
MPI_INFO_NULL, &fh_b);
    MPI_File_read_all(fh_b, 0, b.data(), N, MPI_DOUBLE, MPI_STATUS_IGNORE);
    MPI_File_close(&fh_b);

    std::vector<double> x(N, 0.0);
    std::vector<double> r_local(local_N), Ar_local(local_N), r_global(N);

    double b_norm2 = 0.0;
    for (double val : b) b_norm2 += val * val;
    double r0_norm = std::sqrt(b_norm2);

    for (int iter = 0; iter < MAX_ITER; ++iter)
    {
        // r_local = b_local - (Ax)_local
        for (int i = 0; i < local_N; ++i)
        {
            double sum = 0.0;
            for (int j = 0; j < N; ++j) sum += A_local[i * N + j] * x[j];
            r_local[i] = b[local_start + i] - sum;
        }

        // Собираем полную невязку для следующего шага
        MPI_Allgather(r_local.data(), local_N, MPI_DOUBLE, r_global.data(),
local_N, MPI_DOUBLE, MPI_COMM_WORLD);

        double loc_r2 = 0.0;
```

```

        for (double val : r_local) loc_r2 += val * val;
        double glob_r2;
        MPI_Allreduce(&loc_r2, &glob_r2, 1, MPI_DOUBLE, MPI_SUM,
MPI_COMM_WORLD);

        if (std::sqrt(glob_r2) / r0_norm < EPS) break;

        // Считаем tau
        for (int i = 0; i < local_N; ++i)
        {
            Ar_local[i] = 0.0;
            for (int j = 0; j < N; ++j) Ar_local[i] += A_local[i * N + j] *
r_global[j];
        }

        double loc_rAr = 0.0, loc_ArAr = 0.0;
        for (int i = 0; i < local_N; ++i)
        {
            loc_rAr += r_local[i] * Ar_local[i];
            loc_ArAr += Ar_local[i] * Ar_local[i];
        }

        double g_rAr, g_ArAr;
        MPI_Allreduce(&loc_rAr, &g_rAr, 1, MPI_DOUBLE, MPI_SUM,
MPI_COMM_WORLD);
        MPI_Allreduce(&loc_ArAr, &g_ArAr, 1, MPI_DOUBLE, MPI_SUM,
MPI_COMM_WORLD);

        double tau = g_rAr / g_ArAr;
        for (int j = 0; j < N; ++j) x[j] += tau * r_global[j];
    }

    if (rank == 0)
    {
        std::cout << "MPI Time: " << MPI_Wtime() - t_start << "s\n"
        << "x[0]: " << x[0] << "\n";
    }

    MPI_Finalize();
    return 0;
}

```

Приложение 4. Код скрипта запуска на кластере

```
#!/bin/bash
#PBS -N SLAU_MPI
#PBS -l nodes=2:ppn=12
#PBS -l walltime=01:00:00
#PBS -j oe

cd $PBS_O_WORKDIR

mpiicpc mpi.cpp -o mpi.out
g++ sequential.cpp -o sequential.out

echo "=== Запуск последовательной версии ==="
mpirun -np 1 ./sequential.out > sequential.txt

# Просто запускаем уже скомпилированные программы
for np in 2 4 8 16 24; do
    echo "=== Запуск на $np процессах ==="
    if [ $np -le 12 ]; then
        mpirun -machinefile $PBS_NODEFILE -np $np ./mpi.out > mpi_${np}.txt
    else
        mpirun -machinefile $PBS_NODEFILE -np $np -perhost 12
        ./mpi.out > mpi_${np}.txt
    fi
done
```

Приложение 5. Таблицы

	ВРЕМЯ <i>КОЛЛЕКТИВ</i>	УСКОРЕНИЕ <i>КОЛЛЕКТИВ</i>	ЭФФЕКТИВНОСТЬ <i>КОЛЛЕКТИВ</i>	ВРЕМЯ <i>ПОСЛЕДОВАТЕЛЬНОЙ:</i> 68.0626
2	35.0011	1.945	97.25%	
4	18.1291	3.754	93.85%	
8	9.99143	6.812	85.15%	
16	5.54583	12.273	76.71%	
24	4.41363	15.421	64.25%	

Приложение 6. Графики

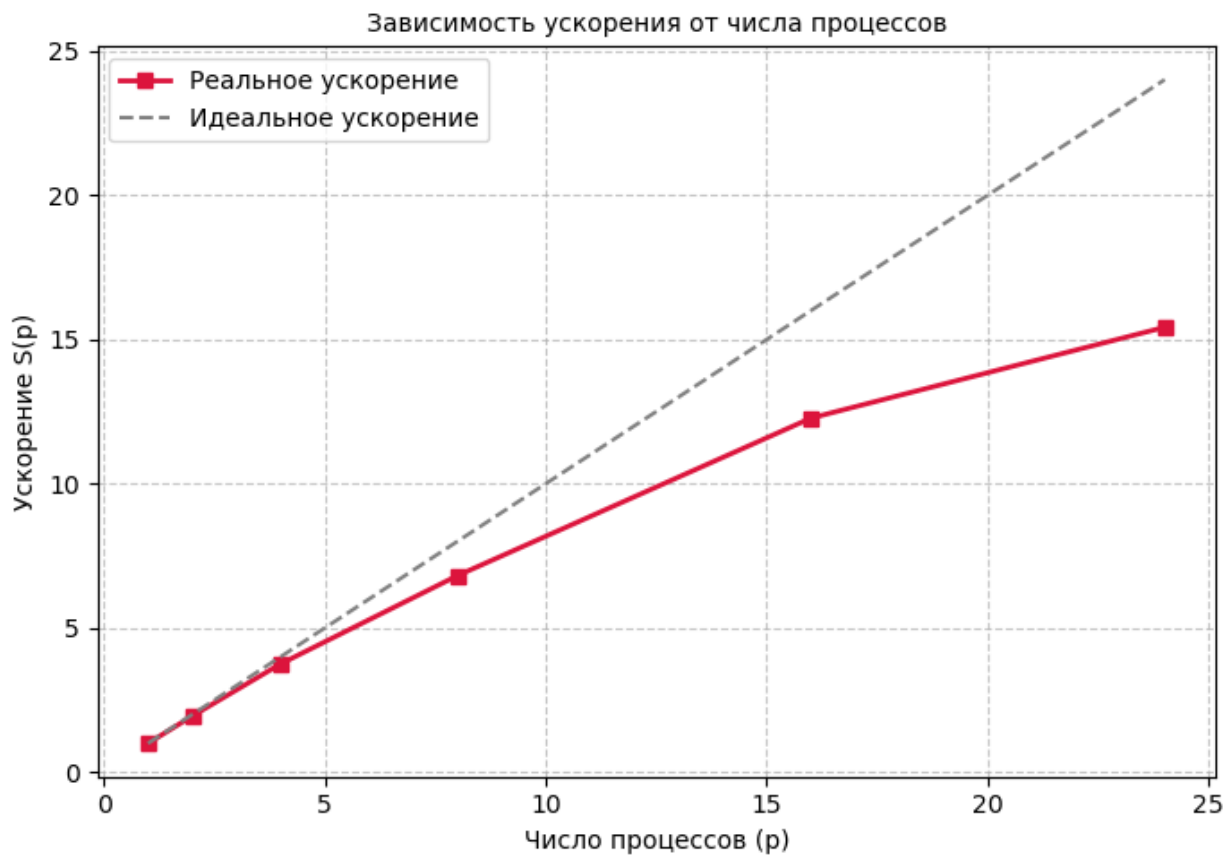
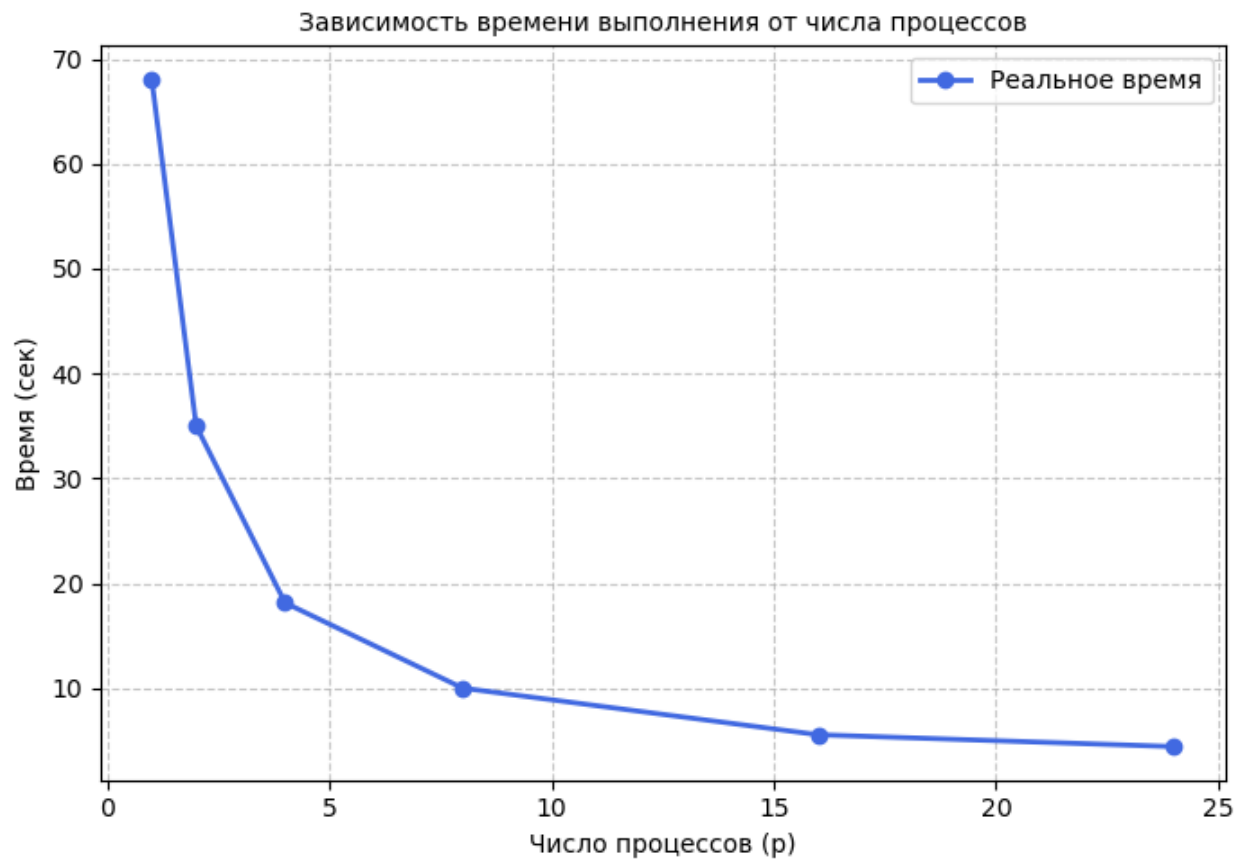
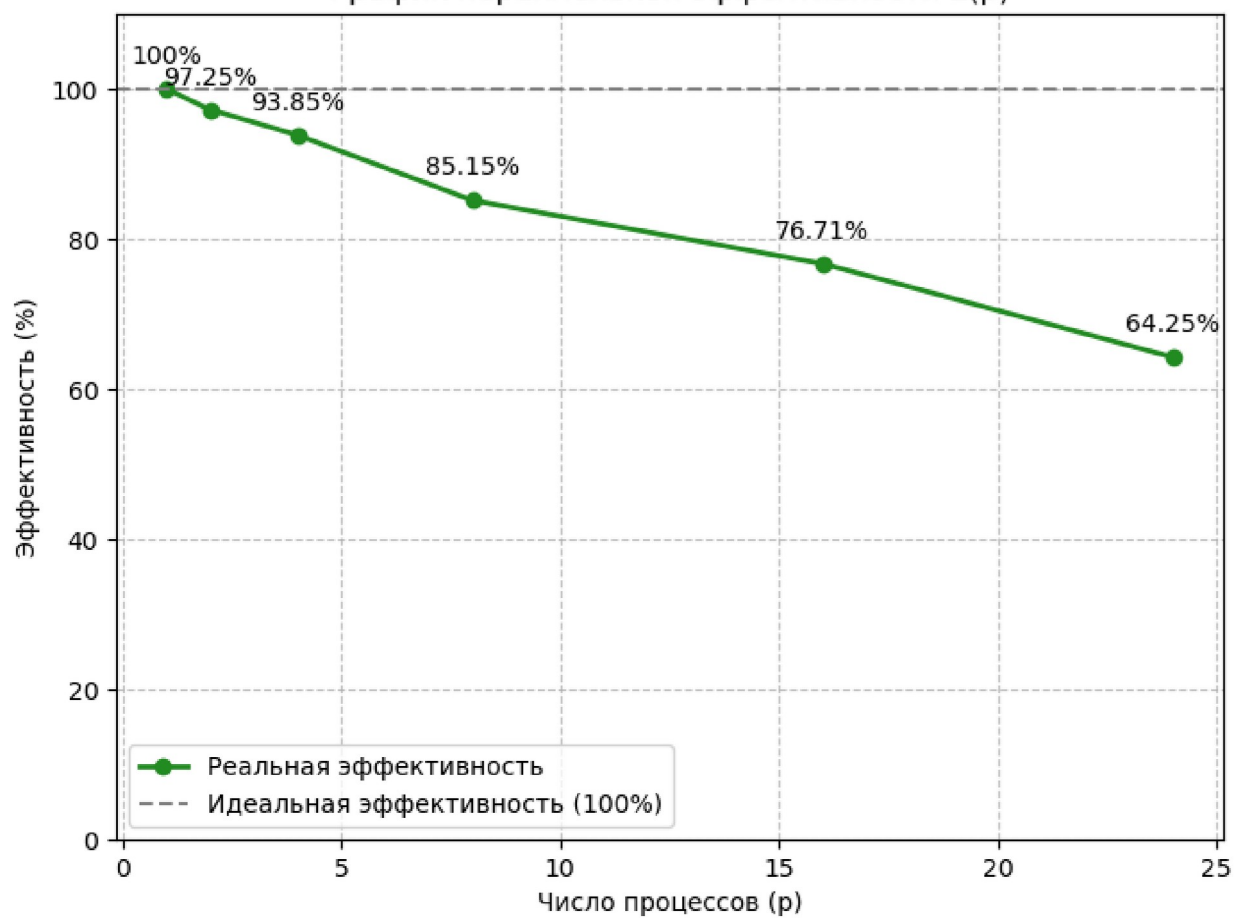


График параллельной эффективности $E(p)$



Приложение 7. Профилирование

